Paper:

Stock markets fluctuate (bear and bull) to reflect the market sentiments and evaluations, and economic activity in general. However, such characteristic behavior increases investment risks making it difficult for institutions and businesses to raise capital. A decrease in investment and economic activity eventually leads to loss of jobs and increased poverty (cheesy line :P). Fund managers use fundamental analysis and technical analysis to quantify these risks and increase investor confidence. While fundamental analysis involves evaluating supply, demand, business models, financial books, etc. underlying the listed company, the technical analysis is restricted to understanding the stock price movements only as a graphical chart to predict its future performance.

With the recent advancements in data science and machine learning, there is growing demand to leverage machine computing power for improving technical analysis (like every other business). Reinforcement Learning (RL) is a suite of machine learning algorithms with an ability to learn (or re-inforce) from near-past and/or expected performances. Consequently, RL is believed to have significant potential to improve technical analysis for stock trades. In the current work, we compare the performance of two different RL algorithms for predicting S&P 500.

RL consists of three components namely, the environment, state and action. The current environment is stock fluctuation or movement of stock price with time. The state is the current situation within the bigger environment where the algorithm has to take a decision (or action) to move to one among several future states without having seen. The action is to buy and sell in order to make profit and loss. Thus, a combination of actions and states results to RL rewards. In order to maximize reward among several such combinations, RL uses a policy framework. In our current work, we compare the performance of RNN and Transformer as deep learning policies.

Gym module from OpenAI was used to set up the trading environment and policy.

RNN neural networks are the simplest form of deep learning algorithms. In simple terms, RNN can be inferred as serial implementation of linear feed forward and activation functions with a hidden layer feedback from previous sequential entities. Due to sequential architecture of RNNs, data positioned later in the input sequence is processed only after the preceding data. In sequential problems such as time series analysis, this helps to preserve the memory and prevents the output to be biased to local or nearest neighbors. Gated neural network (GNN) used in the current project, are a popular version of RNNs. GNNs use update gates and reset gates to store value for a certain period of time and retrieve it as necessary. Consequently, the model learns to decide the amount of information to purge. This prevents vanishing gradient problem of standard RNNs. Details of this concept can be found at https://towardsdatascience.com/what-is-a-recurrent-nns-and-gated-recurrent-unit-grus-ea71d2a05a69

RNN policy was designed to 2 linear layers followed by a convolution layer and GRU (gated recurrent network).

As can be understood, RNNs are difficult to parallelize on GPUs due to their inherent sequential algorithm. Transformers, with their ability to analyse data in parallel offer an excellent alternative. A standard transformer popularly used for NLP problems, primarily contains encoder, decoder, attention layer and positional encoder.

In the current project, transformer decoder was not implemented. In language translation problems, encoder transforms the source language sentence to a pseudo-code and decoder translates this pseudo-code back to the target language for interpretation of end-user. However, for this trading project the end-user does not need translation back of pseudo-code. Encoder outputs can be directly used to take actions in RL environment, rendering decoder redundant.

Another important concept worth mentioning here is positional encoding in transformers. Unlike RNNs, the notion of sequence is not implicitly retained in transformer implementations. Hence, transformers are unable to identify the relative importance of same stock price at different timestamp within the time series. This problem is solved by positional encoding, ie, encoding the entity positions and integrating them with the encoder outputs.

The current transformer was implemented to 2 layers of 2 attention heads with a model embedding dimension of 50.

Results

S&P500 stock prices were divided as train and test data. Train data spanning from 1 January 2014 to 31 August 2018 was used to train RL policies and test data spanning from 1 September 2018 to 31 August 2019 was used to test the model performances. Such a division helps remove performance bias to a particular dataset, that the model may have been exposed while training. The policies were trained for 10 episodes on the training dataset. 10 number of episodes were used in the current code to limit the discussion as a proof of concept. This number can be increased to improve the performance of policy models. However, the number of episodes were not defined for the test dataset. Instead, the policy evaluation was allowed to run indefinitely on test datasets till the policy generated the ratio of portfolio end value to portfolio start value was 1.1. A better policy, will thus, require less episodes to generate similar profits on test dataset.

RNN generated a profit of $797 training in while transformer generated a profit of $1080 on the training data set for similar starting capital of cash and number of shares,for 10 episodes. RNN trained at a rate of 58 episodes/min while transformer trained faster at 18 episodes/min. RNN was able to generate average loss per game of 0.0011% while transformer policy performed better and generated an average profit of 0.0035% per game.

Similarly for the test dataset, RNN generated less profit ($7557) than transformer ($10347) and took more time (586 episodes/min) for prediction as compared to transformer (177 episodes/min). Further, RNN required 11 episodes to achieve the termination criteria (or winning trade with a portfolio_end_value to portfolio_start_value ratio of 1.1) while transformer was able to identify the winning trade in only 2 episodes over the test dataset.

With transformers performing better than RNNs, it can be concluded that transformers are better equipped to learn intrinsic behavior of stock values in a time series. Although the analysis presented here is very limited in its scope for only one entity (S&P 500) and for a defined period, yet the results are very promising to warrant the use of transformers as a potential alternative to RNNs.

Disclaimer:
The above analysis is only for education and research purposes and authors do not claim any liability for its commercial implementation.